

V-Rex: Real-Time Streaming Video LLM Acceleration via Dynamic KV Cache Retrieval

Donghyuk Kim, Sejeong Yang, Wonjin Shin and Joo-Young Kim

KAIST

Daejeon, Republic of Korea

{kar02040, 02yangsj, 2swj1202, jooyoung1203}@kaist.ac.kr

Abstract—Streaming video large language models (LLMs) are increasingly used for real-time multimodal tasks such as video captioning, question answering, conversational agents, and augmented reality. However, these models face fundamental memory and computational challenges because their key-value (KV) caches grow substantially with continuous streaming video input. This process requires an iterative prefill stage, which is a unique feature of streaming video LLMs. Prior works reduce excessive cache overhead by utilizing the KV cache retrieval algorithm, which offloads the full KV cache to CPU memory or storage, then selectively fetches the most relevant entries. Nevertheless, due to its iterative prefill stage, they suffer from significant limitations, including extensive computation, substantial data transfer, and degradation in accuracy. Crucially, this issue is exacerbated for edge deployment, which is the primary target for these models. The memory footprint exceeds the memory capacity within minutes of video streams, making low-latency, energy-efficient inference infeasible.

In this work, we propose V-Rex, the first software-hardware co-designed accelerator that comprehensively addresses both algorithmic and hardware bottlenecks in streaming video LLM inference. At its core, V-Rex introduces ReSV, a training-free dynamic KV cache retrieval algorithm. ReSV exploits temporal and spatial similarity-based token clustering to reduce excessive KV cache memory across video frames, and dynamically adjusts token selection per transformer layer and attention head to minimize the number of selected tokens. To fully realize these algorithmic benefits, V-Rex offers a compact, low-latency hardware accelerator with a dynamic KV cache retrieval engine (DRE), featuring bit-level and early-exit based computing units, as well as hierarchical KV cache memory management. Evaluated on COIN benchmarks, V-Rex achieves unprecedented real-time of 3.9-8.3 FPS and energy-efficient streaming video LLM inference on edge deployment with negligible accuracy loss. While DRE only accounts for 2.2% power and 2.0% area, the system delivers 1.9-19.7 $\times$ speedup and 3.1-18.5 $\times$ energy efficiency improvements over AGX Orin GPU. This work is the first to comprehensively tackle KV cache retrieval across algorithm and hardware, enabling real-time streaming video LLM inference on resource-constrained edge devices, with clear potential for scalable deployment in large-scale server environments.

I. INTRODUCTION

Recently, the demand for artificial intelligence that can understand and interpret various modalities (i.e., text, image, video, and speech) and respond to inquiries has been a driving force in machine learning research. As a result, large multimodal models (LMM) [42] have emerged as promising solutions in various AI industries. Notably, streaming video large language models (LLMs) have gained significant attention for their ability to jointly comprehend the video and text

modalities in real-time. Streaming video LLMs demonstrate a wide range of tasks, including video captioning, question answering, conversational agents, and augmented reality. [4], [32], [38]. Unlike offline video LLMs [29], [33], [45], it processes real-time video streams and responds to users' inquiries, which primarily runs on edge devices. Due to continuous video input requiring an iterative process of video frames, computation and memory usage scale substantially. It causes the key-value (KV) caches to rapidly exceed the GPU memory capacity, and processing long video streams in real-time becomes impractical.

Existing KV cache optimizations are fundamentally mismatched for streaming and interactive workloads. Destructive methods, such as pruning [37], compression [12], [19], [36], and quantization [13], [17], [21], [34] risk permanently discarding information that, while irrelevant to the current query, may be essential for future ones, disrupting conversational continuity. A more promising approach, KV cache retrieval [6], [16], [24], avoids this issue by offloading the full cache to CPU memory or storage and fetching only relevant tokens, thereby reducing GPU memory usage while maintaining coherent responses for more extended input sequences. Although effective in reducing memory usage, they rely on bandwidth-limited links such as PCIe (4–32 GB/s), which are far slower than GPU memory bandwidth (1–2 TB/s). Thus, selective retrieval is necessary to avoid severe data transfer.

However, current retrieval algorithms, designed for the text generation stage, perform poorly under the iterative prefill stage of streaming video. Moreover, their reliance on fixed top-k selection, which is a computationally regular and GPU-friendly primitive, introduces algorithmic inefficiencies. This static strategy ignores the highly variable importance of tokens across transformer layers and attention heads [7], [36], [41]. Enforcing a fixed-k policy prioritizes hardware convenience over the algorithm's need, leading to systemic inefficiencies: either over-fetching redundant tokens, wasting PCIe bandwidth and energy, or under-fetching critical ones, degrading accuracy. Addressing this challenge requires more than an algorithmic tweak. It demands a new hardware optimization.

We present V-Rex, the first streaming video LLM accelerator designed to address the large memory and computational requirements of the KV cache. It embodies this software-hardware co-design principle through the tightly integrated innovations, as shown in Figure 1. At the software level, we

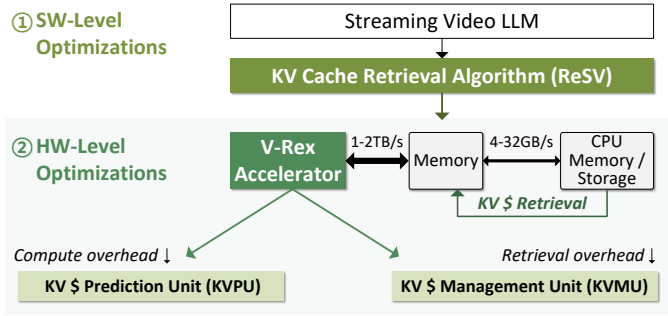


Fig. 1. Overview of V-Rex Accelerator

propose ReSV a training-free KV cache retrieval algorithm that intelligently perceives and exploits the unique characteristics of video data. It significantly reduces the number of fetched tokens for the iterative prefill stage. ReSV’s hash-bit key clustering provides a computationally lightweight mechanism to identify and group tokens with high spatial-temporal similarity, drastically reducing redundancy without expensive computation. Building on this, its weighted cumulative sum (WiCSum) thresholding acts as an adaptive mechanism, dynamically selecting the most critical tokens on a fine-grained, layer-wise, and head-wise basis, moving far beyond the rigid constraints of fixed top-k. At the hardware level, we introduce the dynamic KV cache retrieval engine (DRE), a compact accelerator that serves as the essential enabler for ReSV. The KV cache prediction unit (KVPU) of DRE is specifically designed to execute the fine-grained, data-dependent, and conditional operations of ReSV, such as bit-level clustering and thresholding with early-exit sorting, that would cause severe slowdown and underutilization on a GPU. Additionally, the KV cache management unit (KVMU) of DRE complements this by managing PCIe bandwidth efficiently and reducing overall data movement during retrieval. By offloading these irregular tasks to a specialized unit, V-Rex allows the main LLM engine to operate at peak efficiency.

The key contributions of this work are as follows:

- We propose V-Rex, the first software-hardware co-designed accelerator that fundamentally addresses the large memory and computational bottleneck of the KV cache resulting from the iterative prefill stage in streaming video LLMs.
- We introduce ReSV, a novel, training-free retrieval algorithm tailored for streaming video LLMs that leverages spatial-temporal similarity cache clustering and dynamic cache selection that reduces KV cache traffic with negligible accuracy loss.
- We developed the DRE, an efficient hardware unit that accelerates ReSV’s irregular operations, making intelligent, fine-grained retrieval practical on resource-constrained platforms, consuming only 2.0% of total chip area. It can be integrated with any existing GPUs, NPUs, and LLM accelerators with its high adaptability.
- We demonstrate through comprehensive evaluation that

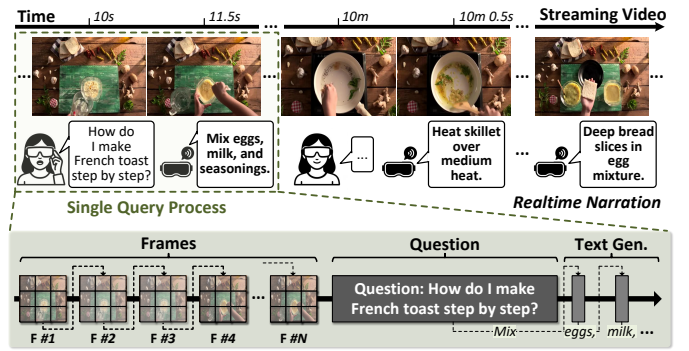


Fig. 2. Overview of Streaming Video LLM

V-Rex enables real-time inference (3.9–8.3 FPS) on edge devices, achieving up to $19.7\times$ speedup and $18.5\times$ energy savings over a state-of-the-art GPU baseline.

II. BACKGROUND AND MOTIVATIONS

A. Streaming Video LLM Architecture and Workflow

Figure 2 presents an overview of the streaming video LLM. Unlike offline models, it processes real-time streaming video input and generates narration in direct response to user queries. Users may issue a series of queries, including follow-ups that depend on both previous interactions and the evolving video context. Consequently, information from earlier video segments is vital for producing context-aware responses to future queries. This operational need underscores the importance of advanced KV cache management algorithms that preserve and utilize prior visual context, rather than relying on conventional optimization methods (i.e., pruning, merging, and quantization) [12], [13], [17], [19], [21], [34], [36], [37] that may discard information essential for subsequent interactions.

Figure 3 shows the model architecture of streaming video LLM. A streaming video LLM consists of three core modules: a vision tower, an MLP projector, and an LLM. The vision tower (e.g., CLIP [27], SigLIP [44]) transforms each video frame into numerical embeddings that capture spatial and temporal features. The MLP projector adapts the dimension of these embeddings, enabling seamless integration with the LLM input space. The LLM processes visual information and user queries, generating output tokens. For the LLM, models such as Llama-3 [8] and Qwen3 [40] are often used.

The streaming video LLM first performs **iterative prefill stage** that sequentially processes video inputs and question tokens, a distinctive mechanism unique to handling real-time video streams. Since sampled frames in a real-time video stream arrive sequentially and cannot be batched together, each frame is processed individually through a repeated prefill computation of LLM. Each prefill stage attends previous KV cache for the self-attention computation and generates KV cache entries that are incrementally accumulated. This KV cache grows linearly over time, following an $O(N^2T)$ complexity, where N^2 represents the spatial resolution and T denotes temporal duration. Notably, this cache facilitates

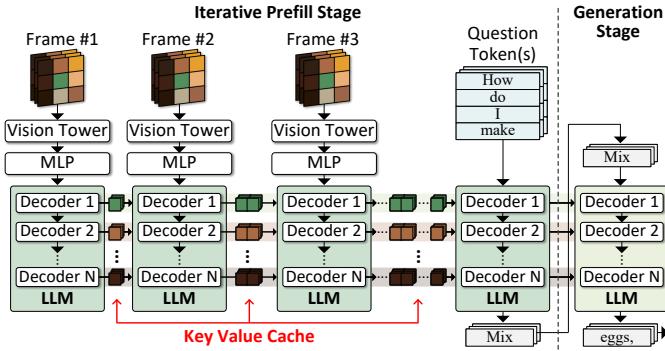


Fig. 3. Model Architecture of Streaming Video LLM

the processing of future frames and is crucial for generating accurate responses to user questions, as queries may reference visual information spanning multiple frames. When the user inputs a query, the user's question is tokenized and processed solely through the LLM. In the generation stage, it generates output based on both the accumulated frame KV caches and the question tokens, thereby maintaining contextual coherence.

B. Principles of KV Cache Retrieval

Figure 4 (a) shows the overhead of the KV cache of VideoLLM-Online [4] when using Llama-3 8B as the backbone model. The KV cache increases with video duration and exceeds GPU memory capacity within minutes, making edge deployment impractical. Prior research attempts to reduce KV cache size through pruning and merging, but it falls short for streaming video LLMs in multi-turn settings. Discarding segments of the cache results in inaccurate responses to sequential user queries. In contrast, KV cache retrieval preserves all prior information and enables selective computation, thereby reducing memory requirements while preserving model accuracy. This is achieved through a three-stage process during inference. (1) Offloading: the entire KV cache is first transferred to CPU memory or storage. (2) Selection: only relevant tokens are selected for the query. (3) Pre-fetching: these selected KV entries are retrieved to the GPU memory in advance for attention computation. This design ensures three essential outcomes: 1) It upholds contextual integrity across multi-turn queries, 2) minimizes the GPU memory requirements, and 3) reduces computation by limiting processing to the most relevant subset of the cache per query. Thus, KV cache retrieval offers both scalability and coherence for real-time streaming video LLMs.

III. CHALLENGES OF KV CACHE RETRIEVALS

A. Why KV Retrievals Fall Short in Streaming Video LLMs

Applying existing KV cache retrieval techniques to streaming video LLMs poses fundamental limitations that have not been addressed in prior works. For instance, InfiniGen [16] is a representative algorithm that effectively hides retrieval latency during the LLM's generation stage. However, in real-world streaming video LLM scenarios, this advantage has minimal

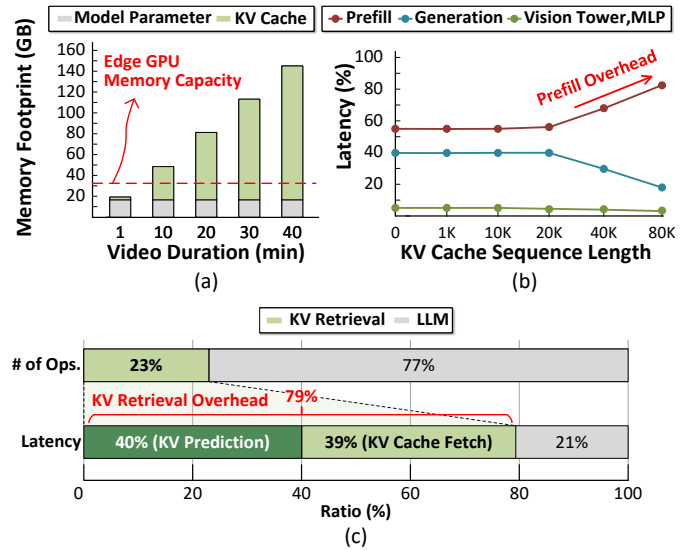


Fig. 4. (a) Memory Footprint of Streaming Video LLM under a 10FPS setting at batch 4. (b) End-to-end Latency Breakdown of Streaming Video LLM. (c) KV Retrieval Latency Overhead of SOTA Retrieval Method [16] in Prefill Stage at 40K KV Cache Sequence Length.

impact because such systems are dominated by an iterative prefill stage, which utilizes KV caches, driven by continuous incoming frames and multi-turn queries. InfiniGen and other similar approaches operate exclusively during generation and thus do not address the predominant bottleneck during prefill, where the bulk of KV cache retrieval and generation occurs.

We analyzed the breakdown of end-to-end latency of streaming video LLM using InfiniGen on an NVIDIA A100 GPU by modeling the average working scenario on the COIN benchmark (i.e., 26 frames, 25 question tokens, and 39 answer tokens), assuming a specific length of the KV cache sequence has been pre-computed and is actively maintained, as shown in Figure 4 (b). The results reveal that as video duration increases, the number of accumulated KV cache tokens grows rapidly, making prefill the largest contributor. At 80K cache sequence length, 83% of the latency is taken by the prefill stage and 74% of this prefill latency is taken by the KV cache retrieval, confirming it is the true bottleneck. Since the prior retrieval method only optimizes the generation stage, it fundamentally fails to tackle the most critical memory and performance bottlenecks in streaming video LLMs. Without directly addressing KV cache management during frequent prefill, it is not possible to achieve practical gains in memory efficiency or system responsiveness under streaming workloads.

B. Limitations of Adapting Retrieval Algorithms

Adapting GPU-oriented retrieval algorithms (FlexGen [30], InfiniGen, ReKV [6]) to streaming video LLM prefill stages causes significant inefficiency due to KV prediction computation and CPU-GPU data transfer overhead. The computation overhead for KV prediction increases as the KV cache sequence increases. In addition, in streaming scenarios, the *Query* matrix consists of multiple tokens, each requiring

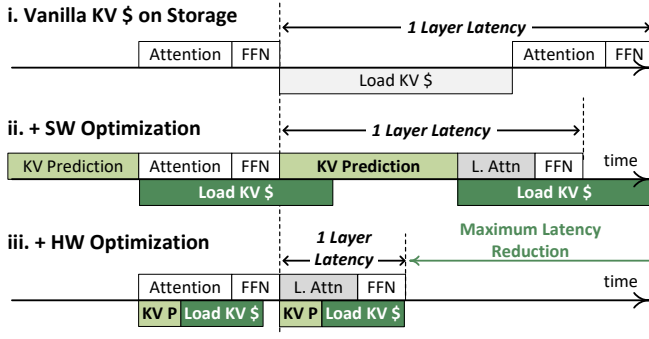


Fig. 5. V-Rex's Software-Hardware Co-design Optimization

different KV cache entries, necessitating larger token budgets than those for text generation. To empirically illustrate these issues, we measure the latency breakdown of streaming video LLM at 40K KV cache sequence length when InfiniGen is adopted for the prefill stage with token budget of 10K on an NVIDIA A100 GPU, as depicted in Figure 4 (c). The KV cache retrieval includes both KV prediction computation and memory transfer latencies. Results show that the KV cache retrieval computation only accounts for 23%. However, it accounts for 85% of the total latency, where 40% is attributed to the KV prediction computation and 39% to the KV cache fetch from CPU memory. We additionally confirmed this issue with other SOTA retrieval methods (i.e., FlexGen and ReKV), demonstrating a similar trend. Furthermore, this issue becomes more severe as the token sequence length increases, causing larger KV prediction computation and data retrieval. These results highlight that existing GPU-oriented algorithms cannot efficiently handle prefill-heavy streaming workloads. Addressing this bottleneck requires fundamentally new approaches.

C. Inflexibility of Fixed Top-K Selection

Many GPU-oriented algorithms, including InfiniGen and ReKV, favor top-k selection in KV cache management to take advantage of the predictable resource allocation and efficient parallel processing inherent to GPU architecture. However, this static approach imposes fundamental limitations for streaming video LLMs. Crucially, the score matrices that determine token importance vary widely across different transformer layers and attention heads, reflecting that diverse features are captured throughout them. Consequently, a different set of tokens is selected as important by each unique layer and head. Prior studies have shown that fixed top-k selection frequently results in redundant tokens or loss of relevant tokens, since the optimal K shifts by layer and head [7], [36], [41].

These inefficiencies are exacerbated in streaming edge environments, where memory budgets are limited and strict latency constraints apply. In such contexts, over-provisioning KV cache due to inflexible top-k policies leads to avoidable resource overhead and longer response times, undermining system scalability and energy efficiency. Additionally, the nature of streaming video LLMs requires the video data to be streamed, and the sequence length increases in real-time,

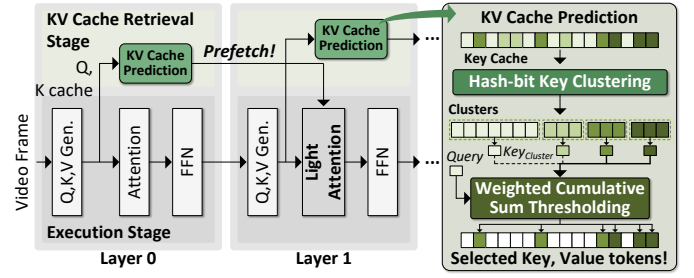


Fig. 6. Overview of ReSV Algorithm

necessitating the adaptive adjustment of the number of selected tokens to ensure efficiency and accuracy. To this end, V-Rex is explicitly designed to address these challenges, providing fine-grained, importance-driven dynamic selection that reduces KV cache size and retrieval cost for practical, real-time inference even on resource-constrained edge platforms.

IV. V-REX: UNIFIED SW-HW CO-DESIGN STRATEGY

To address the challenges of streaming video LLMs, we propose V-Rex, a software-hardware co-designed solution. Figure 5 illustrates how each component of our optimization stack reduces decoder layer latency. At the software level, V-Rex implements ReSV, an enhanced KV cache retrieval algorithm that efficiently selects and fetches only the most relevant entries from CPU memory or storage, where full caches are offloaded. It improves upon prior methods by using hash-bit key clustering and WiCSum thresholding. Leveraging the high temporal and spatial similarity in video frames, the algorithm achieves lightweight computation and efficient KV selection. At the hardware level, V-Rex integrates compact units to accelerate these operations and minimize retrieval overhead. It decouples these operations from the main LLM computation pipeline, enabling latency hiding and end-to-end optimization.

A. ReSV: Efficient and Accurate KV Cache Retrieval

ReSV is a training-free algorithm designed to optimize KV cache retrieval during the iterative prefill stage of streaming video LLMs. As shown in Figure 6, it consists of two main stages: KV retrieval and execution. In the retrieval stage, KV prediction is performed on-the-fly immediately after QKV generation to capture the dynamic nature of the cache. Selected KV tokens are prefetched for the next decoder layer, hiding fetch latency during execution. KV prediction involves two steps. First, hash-bit key clustering groups similar tokens using lightweight bitwise operations, generating representative keys ($Key_{cluster}$) by averaging within each cluster. This reduces computation by limiting attention to representative keys. Second, WiCSum thresholding dynamically selects the most important $Key_{cluster}$, adapting to varying data distributions across layers and attention heads, unlike fixed top-k methods. In the execution stage, the model performs light attention using only the selected clusters, significantly reducing memory and compute by fetching only essential KV entries.

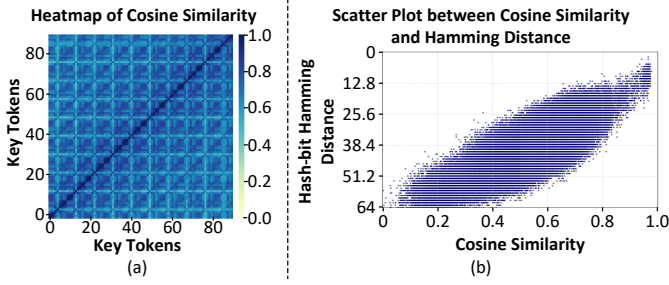


Fig. 7. (a) Heatmap of Cosine Similarity of Key Tokens Between Adjacent Frames (b) Scatter Plot between Cosine Similarity and Hash-bit Hamming Distance. Measured on 3rd Layer's Key using COIN Dataset.

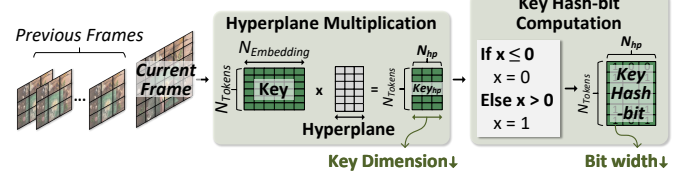
B. Hash-bit Key Clustering for Fast Similarity Grouping

The rationale for hash-bit key clustering lies in the high similarity among tokens in adjacent frames, as shown in Figure 7 (a). Leveraging this property, the method performs spatial-temporal clustering of key caches to efficiently reduce redundancy across frames. Unlike merging methods that replace multiple tokens with a single representation which requires higher precision, this approach preserves original token values for the execution stage. Thus, it avoids expensive operations like high-dimensional cosine similarity by using ultra-low-dimensional representations ($\leq 0.5\%$ of the original dimension) and lightweight hash-bit hamming distance computations. Figure 7 (b) proves that our hash-bit hamming distance can effectively follow the trend of cosine similarity, having a 0.8 correlation value, which is enough to perform clustering.

The clustering process consists of two main steps: hash-bit generation and hamming distance clustering, as shown in Figure 8. In the hash-bit generation step, computation is performed each time a new frame arrives. The key matrix, obtained after applying the rotary position embedding operation to the current frame, undergoes dimensionality reduction in two directions to significantly reduce the overhead of the following hamming distance clustering. A set of N_{hp} random hyperplanes is generated, and the key matrix is multiplied by these hyperplanes, producing a reduced-dimension matrix Key_{hp} with N_{hp} embedding dimensions. Each element of Key_{hp} is then binarized: values less than or equal to zero are set to 0, and values greater than zero are set to 1, converting each element into a single bit to form the *Key hash-bit*.

Next, hamming distance clustering is performed. It involves calculating hamming distance between the newly generated current *Key hash-bit* and the combined *Key_{cluster} hash-bit*, which includes previous and current *Key hash-bits*. The hamming distance is computed by performing a bit-wise XOR operation between tokens and counting the number of differing bits. Tokens with distances below a hyperparameter-defined threshold (Th_{hp}) are clustered. The final clustering results are stored in a hash cluster (HC) table containing the cluster index, token index, *Key_{cluster}*, *Key_{cluster} hash-bit*, and token count. This method enables clustering with very low computational

1. Hash-bit Generation



2. Hamming Distance Clustering

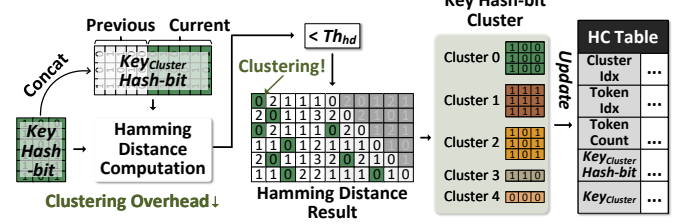
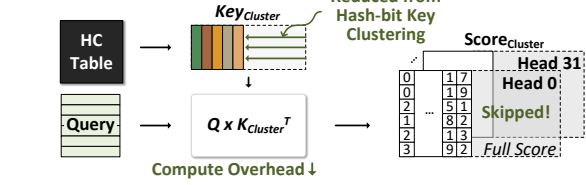


Fig. 8. Dataflow of Hash-bit Key Clustering

3. Query $\times$ Key_{cluster}^T



4. Weighted Cumulative Sum (WiCSum) Thresholding

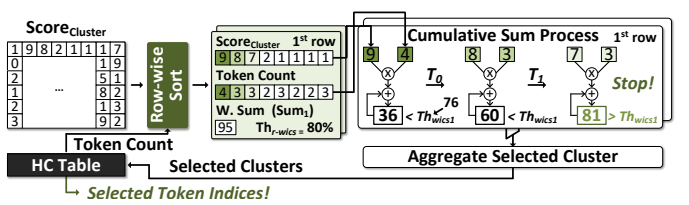


Fig. 9. Dataflow of Weighted Cumulative Sum Thresholding

overhead that typically grows with token count while maintaining high clustering accuracy, making it well-suited for efficient KV cache selection in streaming video LLMs.

C. Dynamic Token Selection via WiCSum Thresholding

WiCSum thresholding is a dynamic selection algorithm developed to address the diverse score distributions that occur across different layers and attention heads. Unlike static top-k methods that select a fixed number of tokens regardless of their importance, WiCSum thresholding dynamically determines the number of tokens to select for each layer and head. This dynamic approach enables more accurate and efficient KV cache retrieval, minimizing unnecessary memory and computational overhead, thereby supporting low latency and high efficiency.

Figure 9 shows the overall dataflow, composed of two main steps: $Query \times Key_{cluster}^T$ computation and threshold checking. In the first step, the algorithm computes the matrix multiplication between the current query vectors and the representative *Key_{cluster}* generated by the previous hash-bit key clustering stage. Because this computation uses only the representative *Key_{cluster}* values rather than the entire key

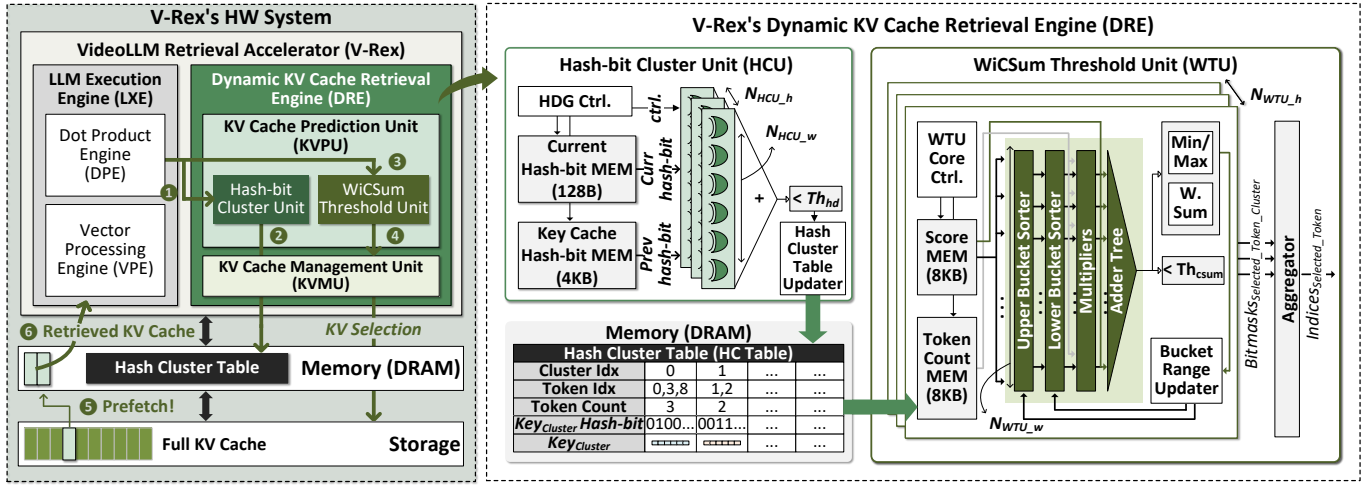


Fig. 10. Overall Architecture of V-Rex

cache, it significantly reduces the computational overhead. The result of this operation is the $Score_{cluster}$ matrix, which reflects the relevance of each $Key_{cluster}$ to the current query.

In the threshold checking step, important elements in the $Score_{cluster}$ matrix are selected. For each row i in the matrix, it calculates a weighted sum (Sum_i) by multiplying each score by its corresponding token count and summing the results, as shown in Equation 1. This weighted sum is then used to compute a threshold (Th_{wics}) by multiplying it by a predefined ratio hyperparameter (Th_{r-wics}), as shown in Equation 2. Then, each row of $Score_{cluster}$ is sorted in descending order, where σ is the sorting permutation. Starting from the highest $Score_{cluster}$ value, the weighted sum with the token count is accumulated until the minimum t , when $Acc_i(t)$ exceeds the threshold Th_{wics} , as shown in Equation 3. The indices of the clusters selected up to this point are aggregated across all rows, and these selected cluster indices are then mapped back to the original token indices using the HC table to produce the final set of selected tokens.

$$Sum_i = \sum_{j=0}^{cluster} Score_{cluster_{i,j}} \cdot TC_j \quad (1)$$

$$Th_{wics_i} = Sum_i \cdot Th_{r-wics} \quad (2)$$

$$Acc_i(t) = \sum_{j=0}^t Score_{cluster_{i,\sigma(j)}} \cdot TC_{\sigma(j)}, Acc_i(t) > Th_{wics_i} \quad (3)$$

V. V-REX'S HARDWARE ARCHITECTURE

The ReSV effectively reduces the number of required tokens. Nevertheless, the core operations introduced by ReSV present inefficiencies on GPUs. These inefficiencies arise from 1) conditional and data-dependent computation of ReSV's clustering and thresholding, which makes it difficult to fully exploit parallelism, and 2) irregular and sparse KV cache fetching from SSD and CPU memory, which causes underutilization of PCIe bandwidth, both resulting in increased

latency. To address these challenges, we introduce V-Rex with a low-latency, compact KV cache retrieval engine specifically designed to support the unique computational patterns of ReSV and optimize the memory-intensive KV fetching by efficiently handling the irregular memory access patterns. Additionally, it can be easily integrated with existing hardware, including GPUs and NPUs, for high adaptability.

A. Architecture Overview

Figure 10 illustrates V-Rex's hardware system, which consists of three primary components: the V-Rex accelerator, off-chip memory, and storage or CPU memory for the full KV cache. The V-Rex accelerator, which comprises the LLM execution engine (LXE) and DRE, is responsible for the primary computational tasks required by streaming video LLMs. The execution flow proceeds as follows: ① LXE generates hash-bits for current frame keys, ② hash-bit cluster unit (HCU) performs hamming distance clustering and updates HC table, ③ LXE computes $Q \times K_{cluster}^T$ then send $Score_{cluster}$ to WiCSum threshold unit (WTU), ④ WTU executes WiCSum thresholding with early-exit sorting, determining which token entries to retrieve, ⑤ KVMU prefetches selected KV entries from storage, and ⑥ retrieved KV tokens are used in attention.

LLM Execution Engine. LXE processes the primary LLM operations and two computations from ReSV. The hash-bit generation and $Query \times Key_{cluster}^T$ computation of ReSV are processed in LXE, as it involves mainly matrix multiplications and vector computations. The LXE is based on the core architecture of the LPU [23], which integrates a dot product engine (DPE) for high-throughput matrix multiplication and a vector processing engine (VPE) for efficient vector operations, both with BF16 precision. DPE is composed of N_{DPE-h} MAC trees, receiving N_{DPE-w} inputs. The VPE is composed of N_{VPE-h} vector units and accepting N_{VPE-w} inputs.

B. Dynamic KV Cache Retrieval Engine (DRE)

The DRE consists of a KVPU and KVMU, which are responsible for the computations and memory management

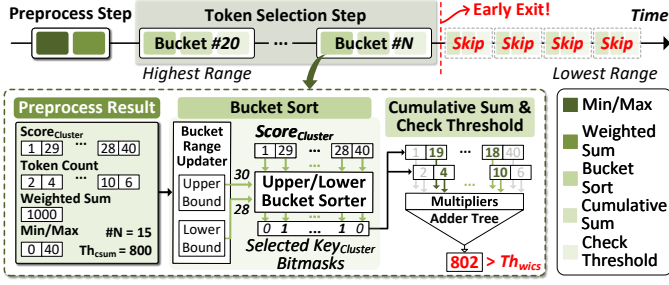


Fig. 11. Dataflow of Early Exit Sorting

required during dynamic KV cache retrieval. The KVPU integrates both HCU and WTU, which together accelerate the most latency-critical operations in KV cache retrieval. By offloading these tasks from the main compute pipeline, V-Rex significantly reduces computational and data fetching bottlenecks.

Hash-bit Cluster Unit. At the core of the KVPU, the HCU is responsible for executing the hash-bit key clustering process. The HCU is a compact computing module, consisting of three main components: a current hash-bit memory, a key cache hash-bit memory, and N_{HCU-h} parallel XOR accumulators, each capable of processing N_{HCU-w} inputs. The HCU receives the key hash-bit vectors from the LXU and stores them in the current hash-bit memory. Simultaneously, it retrieves key cache hash-bit clusters from the HC table and stores them in the key cache hash-bit memory. Both of these are structured as bit matrices to enable efficient parallel operations.

To perform clustering, the HCU initiates the computation of hamming distances between the current hash-bit vectors and the stored key cache hash-bit clusters. This process utilizes XOR accumulators to identify differences between corresponding bits across the matrices. The accumulators then sum the number of differing bits to calculate the hamming distance for each comparison. By comparing the computed hamming distances with the predefined threshold Th_{hd} , the HCU efficiently groups tokens with similar hash-bit patterns into clusters. Then, the clustering results are stored in the HC table. This hardware-accelerated approach enables rapid and energy-efficient clustering using bitwise operators, supporting the low-area requirements for edge deployment.

WiCSum Threshold Unit. The WTU accelerates the WiCSum threshold check, enabling low-latency selection computation. It consists of multiple parallel WTU cores, each equipped with score memory, token count memory, and a dedicated computing unit for thresholding. Each core includes upper and lower bucket sorters, multipliers, an adder tree, and a bucket range updater. The most computationally intensive operations, sorting and threshold checking, are efficiently handled by the WTU's dataflow, which utilizes early exit sorting. It combines two operations in a fine-grained pipeline and terminates sorting in the middle when it exceeds the threshold, as shown in Figure 11. This process is divided into two main steps: the preprocess step and the token selection

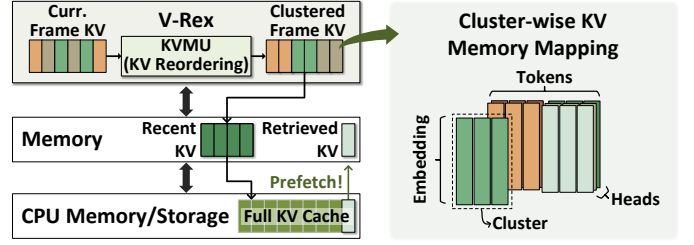


Fig. 12. Hierarchical Memory System and Cluster-wise Memory Mapping

step. In the preprocess step, the WTU cores precompute values needed for the token selection step, such as the weighted sum of scores and token counts for each row, the min/max score values, and the threshold Th_{wics} . During the token selection step, the process begins with the bucket containing the highest score range. The WTU performs bucket sort, cumulative sum, and threshold checking in the pipeline. The bucket sort, which is highly parallelizable, uses the preprocessed information to determine the upper and lower bounds for each bucket, and the sorters generate bitmasks indicating which scores fall within the current range. The selected values are then multiplied and summed to compute the weighted sum, which is compared to Th_{wics} to decide whether to exit or continue. This early exit mechanism is effective because a small number of large scores typically account for the majority of the weighted sum (average 16% per row), allowing the threshold to be reached quickly by starting with the highest buckets.

C. KV Cache Management Unit

The KVMU manages the KV cache's memory for streaming video LLMs. It is responsible for two main functions: hierarchical KV cache memory management and hash cluster-based memory mapping. First, KVMU oversees a hierarchical memory system, as illustrated in Figure 12, to efficiently manage data movement between V-Rex's memory, CPU memory, and storage. Recent KV cache entries are stored in V-Rex's memory for fast access. When the total size of the KV cache in V-Rex's memory exceeds a predefined maximum capacity, the oldest entries are offloaded to CPU memory or storage. These offloaded entries can be retrieved from CPU memory or storage and brought back into V-Rex's memory when needed for computation. This hierarchical memory system ensures that both the most recent and retrieved KV cache entries are available for computation, while older or less critical data is kept off-chip to significantly reduce memory overhead.

Second, KVMU implements an efficient memory mapping strategy based on hash clusters. To maximize PCIe bandwidth utilization, tokens that belong to the same hash cluster are grouped and stored at contiguous memory addresses. The clustering is carried out entirely within the recent KV cache, removing any need to access the CPU or storage for clustering with the offloaded cache. This arrangement enables more efficient use of memory bandwidth, as multiple tokens from the same cluster can be transferred together in a single operation. Each time new KV cache entries are generated for a frame,

KVMU reorders and stores them in memory according to the latest clustering results. Because KVMU handles this reordering internally, the KV cache is stored in a streaming fashion, and any latency associated with reorganization is effectively hidden. Although this memory mapping is technically feasible on conventional GPUs, it is highly impractical because it requires fine-grained, online data reorganization. This process incurs substantial latency overhead that ultimately nullifies the benefits of the optimized layout, as it involves frequent per-layer computations and irregular, memory-intensive scattering operations. To this end, KVMU ensures that streaming video LLMs can access critical cache data with low latency, maintain a reduced memory footprint, and utilize available bandwidth optimally through these two mechanisms.

VI. EVALUATION

A. Experimental Setup

Performance. To evaluate the performance of V-Rex’s hardware system, we developed a custom cycle-level simulator. For DRAM performance, we integrated DRAMSim3 [18], a widely used DRAM simulator, and for SSD performance, we incorporated MQSim [35], an SSD simulator. To accurately model data movement between CPU memory and GPU memory, the actual data transfer bandwidth is modeled using an NVIDIA A100 GPU [3] and an AGX Orin GPU [2], both of which are incorporated into the simulator. We compared V-Rex against two representative GPUs—an edge device (Jetson AGX Orin) and a server GPU (NVIDIA A100)—using identical system and performance parameters, summarized in Table I. For the edge scenario, V-Rex was instantiated with eight cores, utilizing the 4 GB/s PCIe with M.2 NVMe SSD for offloading the KV cache and 204.8 GB/s LPDDR5 of 256-bit bus. For the server scenario, V-Rex utilized 48 cores, achieving a total of 319 TFLOPS, with 1935 GB/s HBM2e of 5120-bit bus and 32 GB/s PCIe with offloading the KV cache to DDR4-based CPU memory. For the streaming video LLM, all experiments employ Llama-3 8B as the backbone model and SigLIP-ViT-L-384 [44] as the vision encoder.

Power/Area. A single V-Rex core is configured as $N_{DPE-h}=64$, $N_{DPE-w}=64$, $N_{VPE-h}=1$, $N_{VPE-w}=64$, $N_{HCU-h}=1$, $N_{HCU-w}=16$, $N_{WTU-h}=1$, and $N_{WTU-w}=16$. It was implemented in RTL and synthesized using Synopsys

Design Compiler on a 14nm process. It operates reliably at 0.8 V and 800 MHz without timing violations under nominal conditions, as confirmed by pre-layout static timing analysis. DRAM (HBM2e, DDR4) behavior was modeled using DRAMSim3, and LPDDR5 energy data were taken from vendor reports [11], [15]. PCIe power was estimated at 3 W per lane under full load, and SSD power was based on Kioxia BG6 specifications [1]. GPU power measurements were obtained via NVIDIA-SMI and tegrastats [25], [26]. All these parameters were integrated into our custom simulator for accurate system-level evaluation. This setup ensures a realistic and fair comparison against baseline edge and server GPUs.

B. Performance and Efficiency Evaluation

Latency. To evaluate V-Rex’s performance for streaming video LLMs, we compared its latency in frame processing and text generation against four top-k-based retrieval methods on both edge and server GPUs. FlexGen [30] serves as the baseline, offloading KV caches to CPU memory (A100) or storage (AGX Orin). InfiniGen [16] retrieves tokens only during generation, InfiniGenP extends this to prefill, and ReKV [6] performs frame-level selection. All baselines conduct KV prediction in the previous attention layer to prefetch KV caches, overlapping fetch latency with computation. We varied KV cache sizes (1K, 5K, 10K, 20K, 40K) using COIN [31], calibrating each method’s selection ratio to match baseline accuracy. Latency was measured as per-frame latency during frame processing and time per output token (TPOT) during text generation.

Latency comparison on the edge GPU is shown in Figure 13 (a). As token length increases, per-frame latency and TPOT rise across all prior methods due to heavier attention computation, greater selection overhead, and larger KV transfers, driven by fixed top-k requiring high token selection ratios. Consequently, none of the edge GPU setups—AGX+FlexGen, InfiniGen, InfiniGenP, or ReKV—achieve real-time performance at longer sequences, with the gap widening as token length grows. In the frame processing stage, the abundance of *Query* tokens demands higher retrieval ratios than in text generation, since each query token requires retrieval. AGX+InfiniGen and AGX+InfiniGenP are even slower than the FlexGen baseline due to fine-grained, token-level selection introducing significant preprocessing overhead. AGX+ReKV’s coarse, frame-level selection offers modest latency gains but still requires selecting many tokens to maintain accuracy, limiting its effectiveness.

In contrast, V-Rex⁸ enables real-time streaming inference (≥ 2 FPS) even with long sequences and large batches. With a batch size of 1, per-frame latencies are 121 ms, 123 ms, 198 ms, 200 ms, and 254 ms for cache sizes of 1K, 5K, 10K, 20K, and 40K, respectively. It maintains 3.9–8.3 FPS across all sizes for real-time edge deployment, achieving $2.2\text{--}7.3\times$ speedups over AGX+FlexGen. When the batch size increases to 4, speedups rise to $2.1\text{--}13.8\times$. In text generation, TPOT latencies are lower, 89 to 97 ms, yielding $1.9\text{--}15.1\times$ speedups. These gains stem from minimizing selected KV volume via

TABLE I
HARDWARE SPECIFICATIONS OF GPUS AND V-REX

	Edge		Server	
	NVIDIA Jetson AGX Orin	V-Rex ⁸	NVIDIA A100	V-Rex ⁴⁸
Number of V-Rex Cores		8		48
Peak Throughput ¹	54	53.3	312	319.5
Memory Bandwidth	LPDDR5 - 204.8 GB/s		HBM2e - 1935GB/s	
Memory Capacity	32GB		80GB	
PCIe Bandwidth	PCIe3.0 x4 4GB/s		PCIe 4.0 x16 32GB/s	
Power Consumption ²	-40W	-35W	-300W	-203.68W

1: FP16 for AGX, BF16 for V-Rex and A100, @ 0.8V 800MHz, 2: V-Rex, DRAM, PCIe, and storage Included

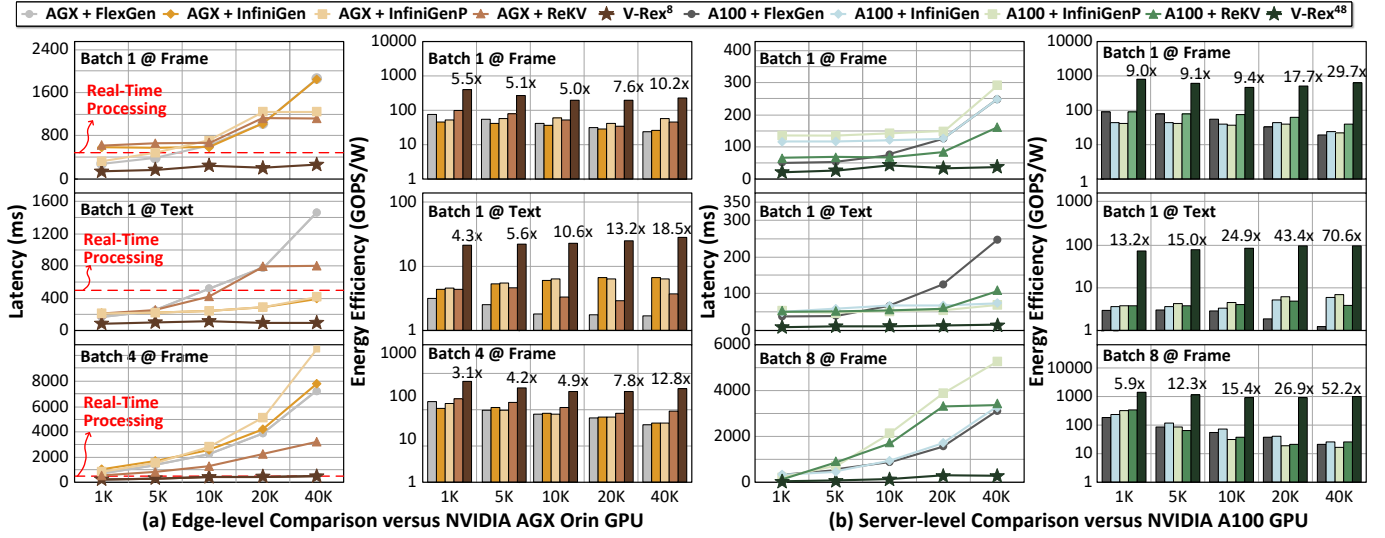


Fig. 13. Per-frame and TPOT latency and energy efficiency comparison versus (a) Edge GPU and (b) Server GPU. Baseline methods of FlexGen, InfiniGen, InfiniGenP, and ReKV are applied. We sweep the KV cache sequence length from 1K to 40K.

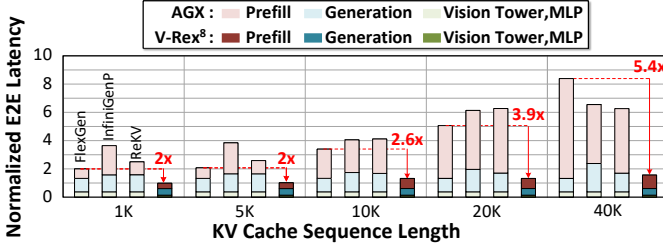


Fig. 14. Comparison of End-to-End Latency Breakdown

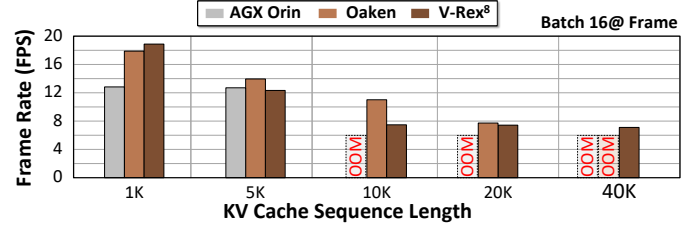


Fig. 15. Throughput Comparison versus SOTA LLM Accelerator

ReSV and leveraging DRE’s high-speed compute and data movement. To evaluate scalability, we tested V-Rex⁴⁸ and an A100 GPU for server-level comparison (Figure 13 (b)). V-Rex achieves 20–48 ms per-frame latency, with 2.6–7.3× speedups at batch size 1. At batch size 8, speedups increase to 3.4–19.7×, demonstrating strong parallel efficiency. TPOT latencies of 14–15 ms yield 2.8–16.8× speedups. These results show that V-Rex significantly reduces latency in both frame processing and text generation for streaming video LLMs over edge and server GPUs.

E2E Latency Breakdown. As shown in Figure 14, we evaluated the latency breakdown of AGX Orin and V-Rex⁸ in an end-to-end streaming video LLM scenario, using an average case from the COIN benchmark. The results demonstrate that AGX+FlexGen fails to mitigate this explosive growth, as well as software-only optimizations (i.e., InfiniGenP and ReKV), which even perform slower than FlexGen from 1K to 20K due to KV prediction overhead. On the other hand, the primary performance gain of our work stems from reducing the overhead of the iterative prefill stage, increasing the performance gap as the KV cache sequence increases. This results in a reduction of up to 5.4× in end-to-end latency. By effectively managing the KV cache during prefill, our method maintains a consistent

latency even as the cache grows.

Energy Efficiency. Figure 13 shows that V-Rex’s energy efficiency gains grow with token length, thanks to reduced data transfer. With batch size 1 during frame processing, V-Rex achieves 5.5–10.2× greater energy efficiency over AGX+FlexGen for KV cache sizes from 1K to 40K; with batch size 4, the gain increases to 3.1–12.8×. In text generation, the improvement is even more pronounced, reaching 4.3–18.5×. This advantage is amplified on server GPUs, where power consumption is higher. Compared to A100+FlexGen, V-Rex achieves 9.0–29.7× higher energy efficiency during frame processing with batch size 1. At batch size 8, it delivers 1.1–1.4 TOPS/W, achieving 5.9–52.2× gains. In text generation, energy efficiency reaches 13.2–70.6×. These improvements stem from two key factors: ReSV’s ability to minimize retrieved data volume, and the KVMU module’s support for high-bandwidth, efficient data fetching. As a result, energy consumption for PCIe-based data transfers is significantly reduced. Overall, V-Rex delivers substantially higher energy efficiency than state-of-the-art GPU-based retrieval methods, making it a compelling solution for streaming video LLM acceleration.

Comparison with SOTA Accelerator. Figure 15 compares the throughput of V-Rex⁸ and Oaken [13], a state-of-the-

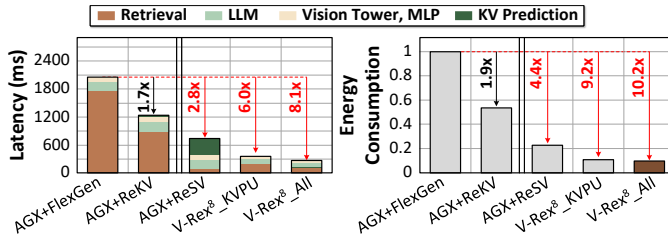


Fig. 16. Ablation Study and Latency Breakdown of V-Rex

art LLM accelerator using 4-bit KV cache quantization, on the NVIDIA AGX Orin GPU. In this setup, AGX Orin runs FlexGen without KV offloading, while Oaken applies online quantization. At a short sequence length (1K), V-Rex achieves up to 1.5 $\times$ and 1.1 $\times$ higher FPS than AGX Orin and Oaken, respectively, due to fully overlapped storage fetches and reduced attention computation. As sequence length increases, AGX Orin encounters out-of-memory (OOM) errors first, driven by the growing KV cache. Oaken, with its quantized cache, handles longer sequences and maintains higher throughput, but still fails beyond 20K tokens due to memory limits. In contrast, V-Rex's efficient retrieval allows it to operate reliably beyond 20K tokens, sustaining 7 FPS even at large sequence lengths.

Ablation Study & Latency Breakdown. This evaluation illustrates how each V-Rex optimization contributes to reducing latency and energy consumption during frame processing. It first presents cumulative gains as each optimization is applied, followed by a latency breakdown showing which execution components are affected by each scheme. We implemented AGX+ReSV by applying ReSV on the AGX Orin GPU and evaluated V-Rex⁸ by incrementally enabling optimizations under a 40K cache with batch size 1. V-Rex⁸_KVPU includes the KVPU, while V-Rex⁸_All incorporates all optimizations, including KVMU. The results clearly demonstrate the GPU's inefficiency and highlight the need for software-hardware co-design.

As shown in Figure 16, AGX+ReSV reduces overall latency by 2.8 $\times$ by hiding most retrieval overhead under computation. However, due to complex KV prediction, such as conditional and data-dependent computation for clustering and thresholding, it still accounts for 48% of total latency, limiting the algorithm's full potential on GPU. With hardware-level optimization, V-Rex⁸_KVPU reduces KV prediction latency overhead down to 0.5% (from 23% of total computation), achieving a 6.0 $\times$ speedup and 9.2 $\times$ energy reduction by overlapping prediction operation with LLM computation using HCU's fast bit-wise operations and WTU's early-exit sorting. V-Rex⁸_All further improves performance by increasing PCIe bandwidth utilization, reaching an 8.1 $\times$ speedup and 10.2 $\times$ energy savings. Although KVMU introduces some memory overhead due to the HC table, it occupies only 1.67% of the full KV cache with an average of 32 tokens per cluster. Each V-Rex optimization contributes incrementally to performance

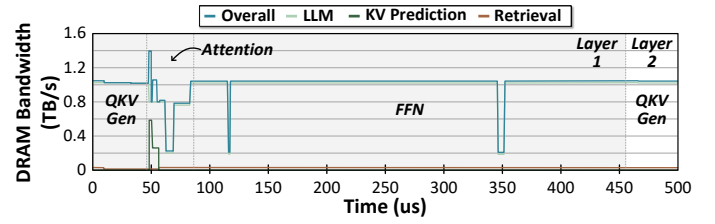


Fig. 17. Analysis on Memory Bandwidth Usage of V-Rex⁸

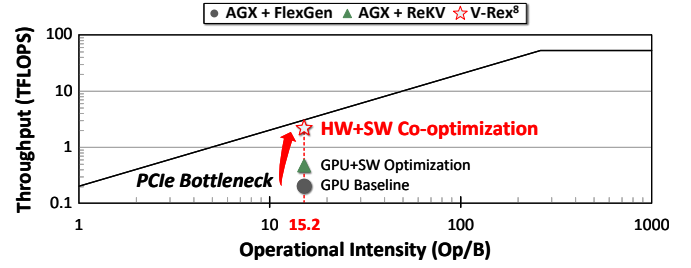


Fig. 18. Roofline Model Analysis on AGX Orin and V-Rex⁸

and energy efficiency. Notably, ReSV alone is insufficient; the combined effect of ReSV and DRE is essential to fully realize efficient KV cache retrieval for streaming video LLMs.

C. Bandwidth Analysis for Concurrent Computation

To show that KV prediction and retrieval can be fully overlapped with LLM computations, we analyzed the bandwidth usage of each computation over a layer of frame processing stage, as shown in Figure 17. It demonstrates that memory is effectively shared among concurrent operations. The KV prediction is executed concurrently with the attention operation. Although it briefly spikes bandwidth usage to 600GB/s, its short duration allows it to be hidden entirely. The KV retrieval, which transfers data from CPU memory to DRAM, runs for most of the execution time but only consumes about 1% of the bandwidth. Because KV cache fetch is bottlenecked by PCIe bandwidth, which is about 1% of DRAM bandwidth, it allows KV retrieval to be concurrently executed with attention and FFN computations with minimal overhead.

D. Roofline Model Analysis

Figure 18 illustrates a roofline model analysis of the frame processing stage for three edge-level systems: AGX+FlexGen, AGX+ReKV, and our proposed V-Rex⁸. This analysis uses a workload with a KV cache length of 40K and a batch size of 4, yielding an average operational intensity of 15.2 Op/B. The result reveals a significant disparity in the achieved throughput across the systems. AGX+FlexGen exhibits the lowest performance, reaching only 6.6% of its theoretical maximum. This severe underutilization is attributed to the slow PCIe communication, which creates a bottleneck during KV cache fetching. Therefore, optimizing the LLM inference computation alone is ineffective without resolving the fundamental I/O bottleneck. AGX+ReKV employs a retrieval mechanism to achieve a higher throughput, reaching approximately

TABLE II
MODEL ACCURACY EVALUATION AND RETRIEVAL RATIO

COIN Benchmark Top-1 Accuracy ↑						
Applied Method	Retrieval @ Frame	Step	Next	Task	Proc.	Proc.+
VideoLLM-Online	X	62.1	49.0	92.5	49.5	51.6
Infinigen	X	62.1	48.3	92.2	49.5	51.0
InfinigenP	O	58.6	45.6	91.5	46.4	50.2
ReKV	O	59.9	46.3	91.3	47.6	50.0
V-Rex's ReSV	O	62.0	47.5	92.2	48.2	50.5

Retrieval Ratio [Frame Processing Stage / Text Generation Stage]						
Applied Method	Avg.	Step	Next	Task	Proc.	Proc.+
Infinigen	100 / 6.8	100 / 6.2	100 / 6.7	100 / 4.0	100 / 8.5	100 / 8.6
InfinigenP	50.8 / 6.8	50.8 / 6.2	50.8 / 6.7	51.0 / 4.0	50.6 / 8.5	50.7 / 8.6
ReKV	58.4 / 31.2	59.7 / 33.4	56.7 / 34.5	51.4 / 13.6	61.7 / 36.7	62.5 / 37.9
V-Rex's ReSV	32.7 / 2.5	34.3 / 2.4	32.4 / 2.8	25.1 / 1.4	35.5 / 2.9	36.1 / 2.9

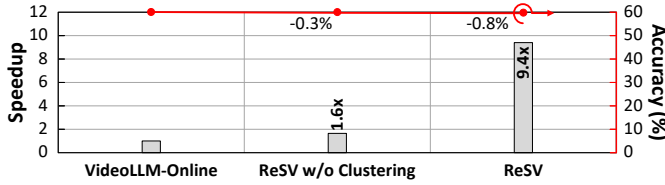


Fig. 19. Ablation Study of ReSV

15% of the peak. However, being a purely software-based optimization, it remains inefficient. Finally, our proposed V-Rex demonstrates a remarkable throughput at 71.5% of its theoretical maximum, marking a 10.8 $\times$ improvement over AGX+FlexGen. It confirms that V-Rex effectively resolves the inefficiencies inherent in conventional GPU-based systems.

E. Comparative Accuracy Analysis

Workload. To demonstrate the flexibility and accuracy of ReSV, we evaluated and compared the performance of existing retrieval methods (i.e., InfiniGen, InfiniGenP, and ReKV) using five benchmarks from the COIN dataset. VideoLLM-Online [4] was used as the baseline without any retrieval optimization applied. For this experiment, existing methods were configured to select up to 50% of tokens with their fixed top-k mechanism, while ReSV used a threshold in its WiCSum operation that was empirically tuned to ensure the accuracy, configuring $N_{hp}=32$, Th_{wics} to 0.3 and $Th_{hp}=7$.

Accuracy. Table II summarizes the results. V-Rex's ReSV outperforms other retrieval methods, demonstrating the lowest retrieval ratio while achieving the highest overall accuracy. Compared to the baseline vanilla model (VideoLLM-Online), ReSV exhibits only a marginal average accuracy drop of 0.8%. Additionally, ReSV significantly reduces the retrieval ratio, as it can adopt diverse score distributions from various tasks. During the frame processing stage, the average retrieval ratio ranges from 25.1% to 36.1%, and during the text generation stage, it varies between 1.4% and 2.9%. This variability highlights that the thresholding mechanism in ReSV effectively adapts token selection according to each task's characteristics.

In contrast, InfiniGen maintains accuracy comparable to the vanilla model, but it does not perform retrieval during

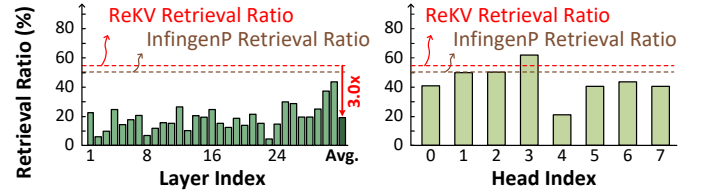


Fig. 20. Comparison of Retrieval Ratio per Layer and per Head

the frame processing stage, making it impractical for real-time inference. InfiniGenP retrieves approximately 50% of tokens during the frame processing stage, which leads to a substantial accuracy degradation of up to 3.4%. ReKV, which employs frame-wise selection, requires a large volume of retrieved KV cache for both frame processing and text generation stages to maintain the accuracy as InfiniGenP, thus degrading the efficiency. In summary, the hash-bit key clustering and WiCSum thresholding techniques of ReSV enable dynamic adaptation to data distribution, effectively selecting the minimal number of tokens while preserving accuracy. This makes ReSV particularly suitable for real-time and resource-constrained streaming video LLM inference.

ReSV Efficiency. We performed an ablation study by incrementally applying ReSV's optimizations. Figure 19 shows the average accuracy on COIN benchmarks and the frame processing latency at 40K cache size. First, ReSV without applying clustering improves latency by 1.6 $\times$ over the baseline, causing a minor accuracy drop of 0.3%, originating from the light attention computation. Second, ReSV, which further incorporates hash-bit clustering, achieves a 9.4 $\times$ speedup, accompanied by a 0.8% accuracy reduction. This significant speedup comes from reducing the fetching and computing of the entire key for WiCSum thresholding computation by clustering the key cache.

Figure 20 presents the ratio of retrieved data on a per-layer and per-head basis of a sample video from COIN. Unlike InfiniGenP and ReKV, which retrieve a fixed number of KV cache tokens uniformly across all layers and heads, ReSV exhibits a diverse distribution in the token retrieval ratio. Certain layers that require fewer tokens exhibit selection rates of 4.2%, while more critical layers with higher token importance demonstrate around 44.0%. This variability can also be observed among the attention heads. It enables ReSV to maintain higher accuracy while retrieving 3.0 $\times$ fewer tokens on average compared to ReKV, resulting in superior efficiency compared to fixed top-k mechanisms.

F. V-Rex's Hardware Overhead

Power and Area. Table III summarizes the power consumption and area breakdown for V-Rex equipped with a single core. A single V-Rex core occupies 1.89 mm² and consumes 2.61 W, equipped with on-chip memory of 384 KB for LXE and 20.125 KB for DRE. When scaled to larger configurations, the area of V-Rex⁸ is 15.12 mm², which is substantially smaller than the AGX Orin GPU (200 mm²). Notably, V-Rex⁴⁸

TABLE III
BREAKDOWN OF AREA AND POWER

Hardware Components		Area[mm ²]		Power [mW]	
LXE	DPE	1.37	72.79%	2311.39	88.58%
	VPE	0.14	7.18%	122.06	4.68%
	On-chip Memory	0.34	17.98%	118.94	4.56%
DRE	KVPU - HCU	0.01	0.28%	2.99	0.11%
	KVPU - WTU	0.02	1.23%	39.04	1.49%
	KVMU	0.01	0.53%	15.01	0.58%
Total		1.89	100%	2609.43	100%

occupies 90.57 mm², considerably less than the A100 GPU (826 mm²). Including overall system power, V-Rex⁸ consumes 35 W, achieving 11.4% lower power consumption than the AGX Orin GPU (40 W), while V-Rex⁴⁸ consumes 203.68 W, demonstrating 32.1% lower power consumption than the A100 GPU (300 W), as detailed in Table I. The additional hardware overhead of DRE is minimal, accounting for only 2.4% of the chip's total power and 2.0% of the total area, which can be attributed to the effective KV cache retrieval algorithm. Its compact design enables efficient integration with any existing GPUs, NPUs, and LLM accelerators.

VII. RELATED WORK & DISCUSSION

Streaming Video LLM Optimization. VideoLLM-Online [4] introduces a streaming video interaction system that adjusts frame sampling and resolution. LiveVLM [24] focuses on reusable short- and long-term memory tokens for efficient frame understanding. QuickVideo [28] leverages parallel CPU decoding and GPU inference overlap to achieve end-to-end speedups. VidMoD [37] reduces visual processing workload using mixture-of-depth computation, dynamically skipping unnecessary layers. While these systems improve frame processing efficiency, they still suffer from repeated KV cache transfers and memory overhead during multi-query streaming workloads. In contrast, V-Rex integrates clustering and dynamic thresholding directly into the prefill stage to manage KV transfers more effectively.

KV Cache Management and Compression. To address the growing memory cost of KV caches, various methods reduce KV size or selectively fetch relevant tokens. LeanKV [46] employs multi-precision KV cache quantization, where high-precision keys and low-precision values are used. Mini-Cache [19] and PyramidInfer [41] compress based on layer-wise importance but leave inter-frame similarity unexplored. CacheGen [20] exploits a traditional video codec mechanism but requires expensive pre-processing overhead for compression, which is not suitable for streaming applications. All these methods use fixed compression and pruning policies that cannot adapt to the evolving attention patterns in video streams. ReSV clusters similar tokens across frames, and its dynamic token selection per layer and head yields more precise retrieval without sacrificing context integrity.

Transformer Hardware Accelerators. Previous works primarily focused on computational and memory bandwidth optimizations in accelerating LLMs. DFX [10] and SpecEE [39]

target to accelerate the memory-bound generation stage. Bit-MoD [5] and LUT Tensor Core [22] reduce computational overhead with a low-bit inference method. While AiF [14] demonstrates in-flash processing for on-device LLMs, it does not incorporate dynamic retrieval mechanisms. Oaken [13] introduces KV cache quantization, which effectively increases the maximum cache capacity but does not address the fundamental structural issue of OOM errors caused by unbounded cache growth. Our method can be applied on top of such prior techniques to further optimize streaming video LLM inference and tackle challenges beyond simple cache size expansion. For vision-oriented models, accelerators such as Adaptiv [43] and EXION [9] primarily focus on improving compute density. However, these designs do not directly address the growth of KV cache overhead unique to streaming video LLM.

VIII. CONCLUSION

We presented V-Rex, the first end-to-end accelerator and KV cache management solution tailored for streaming video LLMs with dynamic KV cache retrieval. Our contributions span the algorithm and hardware levels. Through ReSV, a training-free, dynamic KV retrieval method, V-Rex reduces KV cache volume with negligible accuracy drop. To support ReSV efficiently, we designed a compact hardware unit, DRE, supporting low-latency and energy-efficient computation. V-Rex enables real-time streaming inference, achieving 3.9-8.3 FPS on an edge deployment with 1.9-19.7 $\times$ speedup and 3.1-18.5 $\times$ energy efficiency gains, and extends to 2.6-19.7 $\times$ speedup gains and 5.9-70.6 $\times$ energy efficiency gains over a server GPU. As KV cache retrieval becomes increasingly crucial in long-context and streaming LLMs, we believe V-Rex presents a promising direction for future research in real-time, energy-efficient LLM acceleration, particularly for resource-constrained edge environments and scalable deployment.

ACKNOWLEDGEMENTS

This work was supported by Institute for Information & communications Technology Promotion (IITP) grant funded by the Korea government (MSIT) (No. RS-2025-02264029, Integration and Validation of an AI Semiconductor-Based Data Center Training and Inference System and No. IITP-2025-RS-2023-00256472, Graduate School of Artificial Intelligence Semiconductor).

REFERENCES

- [1] "Bg6 series (m.2) — kioxia - japan," <https://www.kioxia.com/en-jp/business/ssd/client-ssd/bg6.html>, accessed: 2025-07-20.
- [2] "Jetson agx orin for next-gen robotics - nvidia," <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/>, accessed: 2025-07-20.
- [3] "Nvidia a100 tensor core gpu," <https://www.nvidia.com/en-us/data-center/a100/>, accessed: 2025-07-20.
- [4] J. Chen, Z. Lv, S. Wu, K. Q. Lin, C. Song, D. Gao, J.-W. Liu, Z. Gao, D. Mao, and M. Z. Shou, "Videollm-online: Online video large language model for streaming video," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 18 407–18 418.
- [5] Y. Chen, A. F. AbouElhamayed, X. Dai, Y. Wang, M. Andronic, G. A. Constantinides, and M. S. Abdelfattah, "Bitmod: Bit-serial mixture-of-datatype llm acceleration," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1082–1097.

- [6] S. Di, Z. Yu, G. Zhang, H. Li, TaoZhong, H. Cheng, B. Li, W. He, F. Shu, and H. Jiang, "Streaming video question-answering with in-context video KV-cache retrieval," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=8g9fs6mdEG>
- [7] S. Ge, Y. Zhang, L. Liu, M. Zhang, J. Han, and J. Gao, "Model tells you what to discard: Adaptive KV cache compression for LLMs," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=uNrFpDPMYo>
- [8] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Roziere, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. Canton Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Wyatt, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. M. Smith, F. Radenovic, G. Guzmán, F. Zhang, G. Synnaeve, G. Lee, G. L. Anderson, G. Thattai, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. Arrieta Ibarra, I. Kloumann, I. Misra, I. Evtimov, J. Zhang, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van der Linde, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnstun, J. Saxe, J. Jia, K. Vasuden Alwala, K. Prasad, K. Upasani, K. Plawiak, K. Li, K. Heafield, K. Stone, K. El-Arini, K. Iyer, K. Malik, K. Chiu, K. Bhalla, K. Lakhotia, L. Rantala-Yeary, L. van der Maaten, L. Chen, L. Tan, L. Jenkins, L. Martin, L. Madaan, L. Malo, L. Blecher, L. Landzaat, L. de Oliveira, M. Muzzi, M. Pasupuleti, M. Singh, M. Paluri, M. Kardas, M. Tsimpoukelli, M. Oldham, M. Rita, M. Pavlova, M. Kambadur, M. Lewis, M. Si, M. K. Singh, M. Hassan, N. Goyal, N. Torabi, N. Bashlykov, N. Bogoychev, N. Chatterji, N. Zhang, O. Duchenne, O. Çelebi, P. Alrassy, P. Zhang, P. Li, P. Vasic, P. Weng, P. Bhargava, P. Dubal, P. Krishnan, P. Singh Koura, P. Xu, Q. He, Q. Dong, R. Srinivasan, R. Ganapathy, R. Calderer, R. Silveira Cabral, R. Stojnic, R. Raileanu, R. Maheswari, R. Girdhar, R. Patel, R. Sauvestre, R. Polidoro, R. Sumbaly, R. Taylor, R. Silva, R. Hou, R. Wang, S. Hosseini, S. Chennabasappa, S. Singh, S. Bell, S. S. Kim, S. Edunov, S. Nie, S. Narang, S. Rapahty, S. Shen, S. Wan, S. Bhosale, S. Zhang, S. Vandenheide, S. Batra, S. Whitman, S. Sootla, S. Collot, S. Gururangan, S. Borodinsky, T. Herman, T. Fowler, T. Sheasha, T. Georgiou, T. Scialom, and T. Speckbacher, "The Llama 3 Herd of Models," *arXiv e-prints*, p. arXiv:2407.21783, Jul. 2024.
- [9] J. Heo, A. Putra, J. Yoon, S. Yune, H. Lee, J.-H. Kim, and J.-Y. Kim, "Exion: Exploiting inter-and intra-iteration output sparsity for diffusion models," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 324–337.
- [10] S. Hong, S. Moon, J. Kim, S. Lee, M. Kim, D. Lee, and J.-Y. Kim, "Dfx: A low-latency multi-fpga appliance for accelerating transformer-based text generation," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 616–630.
- [11] J. Kim, "The future of graphic and mobile memory for new applications," in *2016 IEEE Hot Chips 28 Symposium (HCS)*. IEEE, 2016, pp. 1–25.
- [12] M. Kim, K. Shim, J. Choi, and S. Chang, "Infinipot-v: Memory-constrained kv cache compression for streaming video understanding," *arXiv preprint arXiv:2506.15745*, 2025.
- [13] M. Kim, S. Hong, R. Ko, S. Choi, H. Lee, J. Kim, J.-Y. Kim, and J. Park, "Oaken: Fast and efficient llm serving with online-offline hybrid kv cache quantization," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 482–497.
- [14] J. Lee, H. Kim, S. Oh, M. Chun, M. Kim, and J. Kim, "Aif: Accelerating on-device llm inference using in-flash processing," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 529–543.
- [15] S. Lee, Y. Ro, Y. H. Son, H. Cho, N. S. Kim, and J. H. Ahn, "Understanding power-performance relationship of energy-efficient modern dram devices," in *2017 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2017, pp. 110–111.
- [16] W. Lee, J. Lee, J. Seo, and J. Sim, "{InfiniGen}: Efficient generative inference of large language models with dynamic {KV} cache management," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 155–172.
- [17] J. Li, Y. Zhang, M. Y. Hassan, T. Chafekar, T. Cai, Z. Ren, P. Guo, F. Karimzadeh, C. Reed, C. Wang, and C. Gan, "CommVQ: Commutative vector quantization for KV cache compression," in *Forty-second International Conference on Machine Learning*, 2025. [Online]. Available: <https://openreview.net/forum?id=sbbyCB39HN>
- [18] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, "Dramsim3: A cycle-accurate, thermal-capable dram simulator," *IEEE Computer Architecture Letters*, vol. 19, no. 2, pp. 106–109, 2020.
- [19] A. Liu, J. Liu, Z. Pan, Y. He, G. Haffari, and B. Zhuang, "Minicache: Kv cache compression in depth dimension for large language models," in *Proceedings of the 38th International Conference on Neural Information Processing Systems*, ser. NIPS '24. Red Hook, NY, USA: Curran Associates Inc., 2025.
- [20] Y. Liu, H. Li, Y. Cheng, S. Ray, Y. Huang, Q. Zhang, K. Du, J. Yao, S. Lu, G. Ananthanarayanan, M. Maire, H. Hoffmann, A. Holtzman, and J. Jiang, "Cachegen: Kv cache compression and streaming for fast large language model serving," in *Proceedings of the ACM SIGCOMM 2024 Conference*, ser. ACM SIGCOMM '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 38–56. [Online]. Available: <https://doi.org/10.1145/3651890.3672274>
- [21] Z. Liu, J. Yuan, H. Jin, S. H. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, "Kivi: a tuning-free asymmetric 2bit quantization for kv cache," in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML/24. JMLR.org, 2024.
- [22] Z. Mo, L. Wang, J. Wei, Z. Zeng, S. Cao, L. Ma, N. Jing, T. Cao, J. Xue, F. Yang, and M. Yang, "Lut tensor core: A software-hardware co-design for lut-based low-bit llm inference," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 514–528. [Online]. Available: <https://doi.org/10.1145/3695053.3731057>
- [23] S. Moon, J.-H. Kim, J. Kim, S. Hong, J. Cha, M. Kim, S. Lim, G. Choi, D. Seo, J. Kim, H. Lee, H. Park, R. Ko, S. Choi, J. Park, J. Lee, and J.-Y. Kim, "A latency processing unit: A latency-optimized and highly scalable processor for large language model inference," *IEEE Micro*, vol. 44, no. 6, pp. 17–33, 2024.
- [24] Z. Ning, G. Liu, Q. Jin, W. Ding, M. Guo, and J. Zhao, "LiveVlm: Efficient online video understanding via streaming-oriented kv cache and retrieval," *arXiv preprint arXiv:2505.15269*, 2025.
- [25] NVIDIA Corporation, "NVIDIA System Management Interface (nvdi-smi)," <https://developer.nvidia.com/system-management-interface>, 2008, accessed: 2025-07-20.
- [26] NVIDIA Corporation, "NVIDIA DRIVE OS 5.1 Linux SDK Documentation," https://docs.nvidia.com/drive/drive_os_5.1.6.1L/nvlib_docs/index.html#page/DRIVE_OS_Linux_SDK_Development_Guide/Utilities/util_tegrastats.html, 2019, accessed: 2025-07-20.
- [27] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. Meila and T. Zhang, Eds., vol. 139. PMLR, 18–24 Jul 2021, pp. 8748–8763. [Online]. Available: <https://proceedings.mlr.press/v139/radford21a.html>
- [28] B. Schneider, D. Jiang, C. Du, T. Pang, and W. Chen, "Quickvideo: Real-time long video understanding with system algorithm co-design," *arXiv preprint arXiv:2505.16175*, 2025.
- [29] X. Shen, Y. Xiong, C. Zhao, L. Wu, J. Chen, C. Zhu, Z. Liu, F. Xiao, B. Varadarajan, F. Bordes, Z. Liu, H. Xu, H. J. Kim, B. Soran, R. Krishnamoorthi, M. Elhoseiny, and V. Chandra, "LongVU: Spatiotemporal adaptive compression for long video-language understanding," in *Forty-second International Conference on Machine Learning*, 2025. [Online]. Available: <https://openreview.net/forum?id=XZC4gs1mf>
- [30] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "Flexgen: High-throughput generative inference of large language models with a single gpu," in *International Conference on Machine Learning*. PMLR, 2023, pp. 31 094–31 116.
- [31] Y. Tang, D. Ding, Y. Rao, Y. Zheng, D. Zhang, L. Zhao, J. Lu, and J. Zhou, "Coin: A large-scale dataset for comprehensive instructional video analysis," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 1207–1216.

- [32] Y. Tang, J. Situ, A. Y. Cui, M. Wu, and Y. Huang, "Llm integration in extended reality: A comprehensive review of current trends, challenges, and future perspectives," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, ser. CHI '25. New York, NY, USA: Association for Computing Machinery, 2025. [Online]. Available: <https://doi.org/10.1145/3706598.3714224>
- [33] Y. Tang, J. Bi, S. Xu, L. Song, S. Liang, T. Wang, D. Zhang, J. An, J. Lin, R. Zhu, A. Vosoughi, C. Huang, Z. Zhang, P. Liu, M. Feng, F. Zheng, J. Zhang, P. Luo, J. Luo, and C. Xu, "Video understanding with large language models: A survey," *IEEE Transactions on Circuits and Systems for Video Technology*, pp. 1–1, 2025.
- [34] K. Tao, H. You, Y. Sui, C. Qin, and H. Wang, "Plug-and-play 1. x-bit kv cache quantization for video large language models," *arXiv preprint arXiv:2503.16257*, 2025.
- [35] A. Tavakkol, J. Gómez-Luna, M. Sadrosadati, S. Ghose, and O. Mutlu, "[MQSim]: A framework for enabling realistic studies of modern {Multi-Queue}-{SSD} devices," in *16th USENIX Conference on File and Storage Technologies (FAST 18)*, 2018, pp. 49–66.
- [36] D. Tu, D. Vashchilenko, Y. Lu, and P. Xu, "VL-cache: Sparsity and modality-aware KV cache compression for vision-language model inference acceleration," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=HMrcv7Q4Ub>
- [37] S. Wu, J. Chen, K. Q. Lin, Q. Wang, Y. Gao, Q. Xu, T. Xu, Y. Hu, E. Chen, and M. Z. Shou, "Videollm-mod: Efficient video-language streaming with mixture-of-depths vision computation," *Advances in Neural Information Processing Systems*, vol. 37, pp. 109 922–109 947, 2024.
- [38] H. Xiong, Z. Yang, J. Yu, Y. Zhuge, L. Zhang, J. Zhu, and H. Lu, "Streaming video understanding and multi-round interaction with memory-enhanced knowledge," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=JbPb6RieNC>
- [39] J. Xu, J. Pan, Y. Zhou, S. Chen, J. Li, Y. Lian, J. Wu, and G. Dai, "Speccee: Accelerating large language model inference with speculative early exiting," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 467–481.
- [40] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu, "Qwen3 technical report," 2025. [Online]. Available: <https://arxiv.org/abs/2505.09388>
- [41] D. Yang, X. Han, Y. Gao, Y. Hu, S. Zhang, and H. Zhao, "Pyramidinfer: Pyramid kv cache compression for high-throughput llm inference," in *ACL (Findings)*, 2024, pp. 3258–3270. [Online]. Available: <https://doi.org/10.18653/v1/2024.findings-acl.195>
- [42] S. Yin, C. Fu, S. Zhao, K. Li, X. Sun, T. Xu, and E. Chen, "A survey on multimodal large language models," *National Science Review*, vol. 11, no. 12, p. nwae403, 2024.
- [43] S. Yoo, H. Kim, and J.-Y. Kim, "Adaptiv: Sign-similarity based image-adaptive token merging for vision transformer acceleration," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 64–77.
- [44] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, "Sigmoid loss for language image pre-training," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2023, pp. 11 975–11 986.
- [45] H. Zhang, X. Li, and L. Bing, "Video-llama: An instruction-tuned audio-visual language model for video understanding," in *EMNLP (Demos)*, 2023, pp. 543–553. [Online]. Available: <https://doi.org/10.18653/v1/2023.emnlp-demo.49>
- [46] Y. Zhang, Y. Hu, R. Zhao, J. C. S. Lui, and H. Chen, "Unifying kv cache compression for large language models with leankv," 2025. [Online]. Available: <https://arxiv.org/abs/2412.03131>